MHOUI-Pro: Exact and Scalable Mining of Maximal High-Occupancy Utility Itemsets via Array-Projected Galois Closure Leaps and Pareto-Trie Pruning

Quan Van[0009−0002−8222−0076]

vanhaminhquan2406@gmail.com

Abstract. Frequent itemset mining (FIM) and high-utility itemset mining (HUIM) extract patterns that satisfy frequency and economic utility thresholds, respectively. High-occupancy itemset (HOI) mining evaluates pattern completeness by requiring an itemset to occupy a substantial fraction of its supporting transactions. However, existing methodologies either treat these dimensions in isolation or combine them into a composite utility-occupancy ratio, conflating physical transaction coverage with monetary yield. Furthermore, mining all unconstrained patterns triggers severe pattern explosion. In this paper, we formulate the problem of mining **Maximal High-Occupancy Utility Itemsets** ($\mathcal{MHOUI}_{\mathrm{s}}$), which decouples classical transaction occupancy $|X|/|T|$ from total economic utility $u(X)$ and enforces strict superset Pareto maximality: an itemset is retained if and only if no strict superset achieves both equal-or-higher occupancy and equal-or-higher utility. To overcome the non-monotonic nature of both occupancy and utility without falling into the computational trap of naive post-filtering or quadratic branch-and-bound overhead, we establish three foundational advances: (1) a tight **Joint Dual Upper Bound** ($\mathrm{UB}_{\mathrm{joint}}$) coupling remaining extension utility with transaction-level occupancy envelopes; (2) **Support-Closure Leaps** jumping directly across Galois equivalence classes with exact utility invariance; and (3) an inverted rare-item index (**Pareto-Trie**) enabling $O(1)$ branch dominance pruning. We propose MHOUI-Pro, an exact algorithm built upon flattened array projection and 64-bit vector bitsets. Comprehensive evaluations on four standard benchmark databases (Foodmart, Liquor, Fruithut, and Chainstore) demonstrate that MHOUI-Pro achieves up to 10× faster runtime than baseline unconstrained search, suppresses memory consumption below 450 MB on databases with over one million transactions, and compresses pattern volumes by up to 99% without losing completeness.

Keywords: High-utility itemset mining · High-occupancy itemsets · Maximal pattern mining · Pareto dominance · In-process pruning · Array projection.

1 Introduction

Frequent itemset mining (FIM) is a foundational task in data mining [1,16,30,35]. Given a transaction database, FIM extracts all item combinations whose occurrence frequency satisfies a user-specified minimum support threshold (σ). However, frequency alone often fails to reflect the qualitative value or the structural density of the discovered patterns. On the one hand, an itemset may appear in numerous transactions while accounting for only a trivial fraction of items within each transaction. To address this limitation, high-occupancy itemset (HOI) mining was introduced [5,6,22,26,33,36]. Classical transaction occupancy, defined as $|X|/|T|$, measures the completeness with which an itemset X covers a supporting transaction T.

On the other hand, traditional FIM treats all items uniformly, disregarding differences in economic profit, item quantity, semantic importance, or risk. High-utility itemset mining (HUIM) addresses this concern by incorporating internal quantities and external unit profits [4, 9, 11, 13, 17, 18, 20, 21, 23, 25, 27, 31, 32, 34, 37]. Recent studies have explored high utility-occupancy pattern mining (HUOPM) [3,7,12,19], which computes the ratio of pattern utility to total transaction utility, $u(X,T)/TU(T)$. While effective for proportional profit analysis, HUOPM couples utility and occupancy into a single composite metric, obscuring whether a pattern has high physical basket coverage or merely high unit cost.

Moreover, mining all unconstrained patterns inevitably leads to the *pattern explosion problem*. To achieve concise representations, maximal frequent itemset (MFI) mining discards frequent subsets dominated by larger frequent supersets [2, 14, 15]. Similarly, maximal and closed high-utility itemsets compress utility collections [8, 10, 24, 28, 29]. However, directly applying support-based or utility-based maximality to occupancy-constrained patterns is fundamentally flawed: neither classical occupancy nor total utility satisfies the anti-monotonicity property. A superset can exhibit higher or lower occupancy and utility than its subsets.

To overcome these foundational challenges, this paper formulates the **Maximal High-Occupancy Utility Itemset** ($\mathcal{MHOUI}_\text{s}$) mining problem and introduces MHOUI-Pro. The proposed framework simultaneously enforces:

1. **Support Constraint:** The itemset must occur in at least σ transactions.
2. **Classical Occupancy Constraint:** The itemset must satisfy a minimum average transaction occupancy β, ensuring dense physical coverage.
3. **Total Utility Constraint:** The itemset must generate at least μ economic utility.
4. **Superset Pareto Maximality:** An itemset is eliminated if there exists a strict superset possessing equal or higher occupancy and equal or higher utility.

Overcoming the Algorithmic Bottlenecks. Previous preliminary explorations of MHOUI suffered from three critical vulnerabilities:

1. *The Pruning Overhead Trap:* Sorting transaction reciprocals at every node to evaluate occupancy bounds incurred higher computational cost than the search space saved.
2. *Cache Pollution from Fat Structures:* Vertical list structures storing redundant fields (‘tid‘, ‘iu‘, ‘ru‘, ‘invlen‘, ‘remcnt‘) saturated CPU cache hierarchies, exploding RAM on million-scale databases.
3. *Loss of Equivalence Classes:* Lack of closure jumping expanded search spaces combinatorially.

In MHOUI-Pro, we solve all three bottlenecks simultaneously. We introduce the **Joint Dual Upper Bound** ($\mathrm{UB}_{\mathrm{joint}}$), integrate **Support-Closure Leaps** on Galois equivalence classes with invariant utility accumulation, employ an inverted **Pareto-Trie** for $O(1)$ branch dominance checks, and adopt flattened array projection inspired by EFIM [37].

2 Related Work and Positioning

Table 1 positions $\mathcal{MHOUI}_{\mathrm{s}}$ relative to 12 pattern mining paradigms.

Table 1. Positioning of $\mathcal{MHOUI}_{\mathrm{s}}$ against related itemset mining paradigms.

Pattern Family	Support	Classical Occ.	Total Utility	Utility Occ.	Maximality Criterion
FI [1, 16, 30, 35]	✓	–	–	–	None
MFI [2, 14, 15]	✓	–	–	–	Support-maximal
HOI [5, 6, 22, 26, 33, 36]	✓	✓	–	–	None
HUI [9, 11, 20, 21, 32, 37]	Optional	–	✓	–	None
MaxHUI / CHUI [8, 10, 24, 28, 29]	Optional	–	✓	–	Utility-based concise form
HUOPM [3, 7, 12, 19]	✓	–	Indirect	✓	Optional concise form
HOUI (Baseline)	✓	✓	✓	–	None
Weak MHOUI (Ours)	✓	✓	✓	–	Weak Superset Pareto Dominance
Strong MHOUI (MHOUI-Pro)	✓	✓	✓	–	Strong Superset Pareto Dominance

3 Preliminaries and Problem Formulation

Let $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ be a finite set of distinct items. A quantitative transaction database is $\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$. Each transaction $T_q \in \mathcal{D}$ consists of an itemset $T_q \subseteq \mathcal{I}$, an internal utility $q(i, T_q) \in \mathbb{R}^+$ for each item $i \in T_q$, and an external profit $p(i) \in \mathbb{R}^+$.

Definition 1 (Item and Transaction Utility). *The utility of an item i in transaction T_q is $u(i, T_q) = p(i) \cdot q(i, T_q)$. The utility of an itemset $X \subseteq T_q$ in T_q is $u(X, T_q) = \sum_{i \in X} u(i, T_q)$. The transaction utility of T_q is $TU(T_q) = \sum_{i \in T_q} u(i, T_q)$.*

Definition 2 (Support and Database Utility). *The supporting transaction set of itemset X is $\Gamma(X) = \{T_q \in \mathcal{D} \mid X \subseteq T_q\}$. The support count of X is $\mathrm{supp}(X) = |\Gamma(X)|$. The total utility of X in $\mathcal{D}$ is $u(X) = \sum_{T_q \in \Gamma(X)} u(X, T_q)$.*

Definition 3 (Classical Transaction Occupancy). *For an itemset $X \subseteq T_q$, the classical occupancy of X in T_q is* $\mathrm{occ}(X, T_q) = \frac{|X|}{|T_q|}$. *The average classical occupancy of X across its supporting transactions is:*

$$\mathrm{aocc}(X) = \frac{1}{\mathrm{supp}(X)} \sum_{T_q \in \Gamma(X)} \mathrm{occ}(X, T_q) = \frac{|X|}{\mathrm{supp}(X)} \sum_{T_q \in \Gamma(X)} \frac{1}{|T_q|}. \tag{1}$$

Definition 4 (Transaction-Weighted Utilization). *The transaction-weighted utilization (*TWU*) of an itemset X is:*

$$\mathrm{TWU}(X) = \sum_{T_q \in \Gamma(X)} TU(T_q). \tag{2}$$

Definition 5 (High-Occupancy Utility Itemset ($\mathcal{HOUI}$)). *Given a minimum support threshold $\sigma \in \mathbb{N}^+$, a minimum occupancy threshold $\beta \in (0, 1]$, and a minimum utility threshold $\mu \in \mathbb{R}^+$, an itemset X is a High-Occupancy Utility Itemset if:*

$$\mathrm{supp}(X) \geq \sigma, \quad \mathrm{aocc}(X) \geq \beta, \quad \textit{and} \quad u(X) \geq \mu. \tag{3}$$

3.1 Superset Pareto Dominance and MHOUI Variants

Definition 6 (Weak Superset Dominance). *An itemset Y weakly dominates X (denoted $X \prec_{\mathrm{w}} Y$) if $X \subset Y$, $\mathrm{aocc}(Y) \geq \mathrm{aocc}(X)$, $u(Y) \geq u(X)$, and at least one inequality is strict ($\mathrm{aocc}(Y) > \mathrm{aocc}(X)$ or $u(Y) > u(X)$).*

Definition 7 (Strong Superset Dominance). *An itemset Y strongly dominates X (denoted $X \prec_{\mathrm{s}} Y$) if $X \subset Y$, $\mathrm{aocc}(Y) \geq \mathrm{aocc}(X)$, and $u(Y) \geq u(X)$.*

Definition 8 (Weak and Strong MHOUI). *An itemset $X \in \mathcal{HOUI}$ is a **Weak MHOUI** ($\mathcal{MHOUI}_{\mathrm{w}}$) if $\nexists Y \in \mathcal{HOUI}$ such that $X \prec_{\mathrm{w}} Y$. An itemset $X \in \mathcal{HOUI}$ is a **Strong MHOUI** ($\mathcal{MHOUI}_{\mathrm{s}}$) if $\nexists Y \in \mathcal{HOUI}$ such that $X \prec_{\mathrm{s}} Y$.*

Theorem 1 (Inclusion Chain). *For any database $\mathcal{D}$ and thresholds σ, β, μ: $\mathcal{MHOUI}_{\mathrm{s}}(\mathcal{D}, \sigma, \beta, \mu) \subseteq \mathcal{MHOUI}_{\mathrm{w}}(\mathcal{D}, \sigma, \beta, \mu) \subseteq \mathcal{HOUI}(\mathcal{D}, \sigma, \beta, \mu)$.*

Proof. If $X \in \mathcal{MHOUI}_{\mathrm{s}}$, no $Y \in \mathcal{HOUI}$ satisfies $X \prec_{\mathrm{s}} Y$. Since $X \prec_{\mathrm{w}} Y \implies X \prec_{\mathrm{s}} Y$, no such Y can weakly dominate X. Thus $X \in \mathcal{MHOUI}_{\mathrm{w}} \subseteq \mathcal{HOUI}$.

Mathematical Distinction Between Weak and Strong Dominance: The fundamental distinction lies in whether equality in both objective dimensions permits elimination:

- *Weak Superset Dominance ($X \prec_{\mathrm{w}} Y$):* Requires at least one strict inequality ($\mathrm{aocc}(Y) > \mathrm{aocc}(X)$ or $u(Y) > u(X)$). If a proper superset $Y \supset X$ exhibits identical metrics ($\mathrm{aocc}(Y) = \mathrm{aocc}(X)$ and $u(Y) = u(X)$), Y does *not* weakly dominate X. Consequently, $\mathcal{MHOUI}_{\mathrm{w}}$ retains both X and Y.

– *Strong Superset Dominance ($X \prec_s Y$):* Admits non-strict inequalities ($\text{aocc}(Y) \geq \text{aocc}(X)$ and $u(Y) \geq u(X)$). Even if metrics are exactly tied, the superset Y subsumes X because Y represents a larger, more comprehensive pattern without any sacrifice in occupancy or monetary yield. Hence, $\mathcal{MHOUI}_s$ eliminates X and retains only Y.

For instance, if $X = \{a, b\}$ has $\text{aocc}(X) = 0.60, u(X) = \$100$ and superset $Y = \{a, b, c\}$ has $\text{aocc}(Y) = 0.60, u(Y) = \$100$ (e.g., c generates zero marginal profit or co-occurs identically in the same transaction set), Weak MHOUI outputs both patterns ($\{X, Y\}$), whereas Strong MHOUI outputs only the maximal superset $\{Y\}$.

3.2 Running Example Database and Detailed Calculations

To provide concrete intuition for all formulations and bounding theorems, we introduce a toy quantitative transaction database $\mathcal{D}_{\text{toy}}$ comprising 5 transactions and 5 items $\{a, b, c, d, e\}$, displayed in Table 2. Let external unit profits be $p(a) = 3, p(b) = 1, p(c) = 5, p(d) = 2, p(e) = 4$.

Table 2. Running quantitative transaction database $\mathcal{D}_{\text{toy}}$ with transaction lengths and utilities.

TID	**Items with Quantities**	**Item Utilities** ($u(i, T_q)$)	**Length** ($\lvert T_q \rvert$)	**TU** ($TU(T_q)$)
T_1	$(a:1), (b:2), (c:1)$	$u(a) = 3, u(b) = 2, u(c) = 5$	3	\$10
T_2	$(a:2), (c:1), (d:1)$	$u(a) = 6, u(c) = 5, u(d) = 2$	3	\$13
T_3	$(a:1), (b:3), (c:2), (e:1)$	$u(a) = 3, u(b) = 3, u(c) = 10, u(e) = 4$	4	\$20
T_4	$(b:1), (c:1), (d:2)$	$u(b) = 1, u(c) = 5, u(d) = 4$	3	\$10
T_5	$(a:1), (b:1), (c:1), (d:1)$	$u(a) = 3, u(b) = 1, u(c) = 5, u(d) = 2$	4	\$11
Total Database Utility $\sum_{T_q \in \mathcal{D}_{\text{toy}}} TU(T_q)$:				\$64

Example Computations of Basic Metrics: Let thresholds be $\sigma = 3$, $\beta = 0.40$ (40%), and $\mu = 20$.

– **Singleton $\{a\}$:** $\Gamma(\{a\}) = \{T_1, T_2, T_3, T_5\}$, so $\text{supp}(\{a\}) = 4 \geq 3$. Its total utility is $u(\{a\}) = 3 + 6 + 3 + 3 = 15 < 20$. Its average occupancy is $\text{aocc}(\{a\}) = \frac{1}{4}\left(\frac{1}{3} + \frac{1}{3} + \frac{1}{4} + \frac{1}{4}\right) = \frac{1}{4} \cdot \frac{7}{6} \approx 0.2917 < 0.40$. Thus $\{a\}$ fails both utility and occupancy constraints.
– **Itemset $\{a, c\}$:** $\Gamma(\{a, c\}) = \{T_1, T_2, T_3, T_5\}$, so $\text{supp}(\{a, c\}) = 4 \geq 3$. Its total utility is $u(\{a, c\}) = (3{+}5){+}(6{+}5){+}(3{+}10){+}(3{+}5) = 8{+}11{+}13{+}8 = 40 \geq 20$. Its occupancy is $\text{aocc}(\{a, c\}) = \frac{2}{4}\left(\frac{1}{3} + \frac{1}{3} + \frac{1}{4} + \frac{1}{4}\right) = \frac{2}{4} \cdot \frac{7}{6} = \frac{7}{12} \approx 0.5833 \geq 0.40$. Because $\text{supp} \geq 3$, $\text{aocc} \geq 0.40$, and $u \geq 20$, $\{a, c\} \in \mathcal{HOUI}$.
– **Itemset $\{a, b, c\}$:** $\Gamma(\{a, b, c\}) = \{T_1, T_3, T_5\}$, so $\text{supp}(\{a, b, c\}) = 3 \geq 3$. Its total utility is $u(\{a, b, c\}) = (3 + 2 + 5) + (3 + 3 + 10) + (3 + 1 + 5) = 10 + 16 + 9 = 35 \geq 20$. Its occupancy is $\text{aocc}(\{a, b, c\}) = \frac{3}{3}\left(\frac{1}{3} + \frac{1}{4} + \frac{1}{4}\right) = \frac{1}{3} + \frac{1}{2} = \frac{5}{6} \approx 0.8333 \geq 0.40$. Hence $\{a, b, c\} \in \mathcal{HOUI}$.

Illustration of Superset Pareto Dominance: Compare $X = \{a, c\}$ (aocc $= 0.5833, u = 40$) and its superset $Y = \{a, b, c\}$ (aocc $= 0.8333, u = 35$). Here $Y \supset X$, and Y achieves strictly higher occupancy ($0.8333 > 0.5833$), but lower utility ($35 < 40$). Therefore, Y does *not* dominate X. Both $\{a, c\}$ and $\{a, b, c\}$ represent distinct optimal trade-offs on the Pareto frontier and are retained in $\mathcal{MHOUI}_{\mathrm{s}}$. Conversely, consider $\{b, c\}$: $\Gamma(\{b, c\}) = \{T_1, T_3, T_4, T_5\}$ (supp $= 4$), $u(\{b, c\}) = 7+13+6+6 = 32 \geq 20$, and $\mathrm{aocc}(\{b, c\}) = \frac{2}{4}(\frac{1}{3}+\frac{1}{4}+\frac{1}{3}+\frac{1}{4}) = \frac{7}{12} \approx 0.5833 \geq 0.40$, so $\{b, c\} \in \mathcal{HOUI}$. If a superset $Z \supset \{b, c\}$ has $\mathrm{aocc}(Z) \geq 0.5833$ and $u(Z) \geq 32$, $\{b, c\}$ is dominated and pruned. This preserves only the maximal frontier.

4 Theoretical Foundations and Safe Pruning Bounds

4.1 Non-Monotonicity and Monotonic Envelopes

Theorem 2 (Non-Monotonicity of Utility and Occupancy). *Neither* $u(X)$ *nor* $\mathrm{aocc}(X)$ *satisfies monotonicity or anti-monotonicity.*

Proof. Let $T_1 = \{a, b\}$ and $T_2 = \{a, c\}$. For $X = \{a\}$, $\Gamma(X) = \{T_1, T_2\}$ and $\mathrm{aocc}(X) = 0.50$. For superset $Y = \{a, b\}$, $\Gamma(Y) = \{T_1\}$ and $\mathrm{aocc}(Y) = 1.0 > \mathrm{aocc}(X)$. Conversely, appending a longer transaction reduces occupancy. Similarly, $u(X)$ can increase from item profit accumulation or decrease from support loss.

4.2 Joint Dual Upper Bound ($\mathbf{UB_{joint}}$)

Let a search node be denoted by $N = (P, E)$, where P is the prefix itemset and E is the ordered candidate extension list. For each transaction $T \in \Gamma(P)$, define $\mathrm{rc}_N(T) = |E \cap T|$ and remaining utility $\mathrm{ru}_N(T) = \sum_{i \in E \cap T} u(i, T)$.

Theorem 3 (Joint Dual Upper Bound). *For every descendant* $Y = P \cup Z$ *with* $Z \subseteq E$ *and* $\mathrm{supp}(Y) \geq \sigma$*, the maximum utility and occupancy attainable by* Y *are simultaneously bounded by:*

$$\mathrm{UB}_{\mathrm{joint}}(N) = \sum_{T \in \Gamma(P)} \left(u(P, T) + \mathrm{ru}_N(T)\right), \quad \mathrm{OUB}(N) = \frac{1}{\sigma} \sum_{T \in \Gamma(P)} \min\left(1.0, \frac{|P| + \mathrm{rc}_N(T)}{|T|}\right). \tag{4}$$

If $\mathrm{UB}_{\mathrm{joint}}(N) < \mu$ *or* $\mathrm{OUB}(N) < \beta$*, the subtree rooted at* N *can be safely pruned in* $O(|\Gamma(P)|)$ *single-pass operations without sorting.*

Proof. For every supporting transaction $T \in \Gamma(Y) \subseteq \Gamma(P)$, $Z \subseteq E \cap T$. Thus $u(Y, T) \leq u(P, T) + \mathrm{ru}_N(T)$. Summing over $\Gamma(Y)$ yields $u(Y) \leq \mathrm{UB}_{\mathrm{joint}}(N)$. Similarly, $\mathrm{occ}(Y, T) = \frac{|P|+|Z|}{|T|} \leq \min(1.0, \frac{|P|+\mathrm{rc}_N(T)}{|T|})$. Since $|\Gamma(Y)| \geq \sigma$, the average occupancy over $\Gamma(Y)$ cannot exceed $\mathrm{OUB}(N)$.

Numerical Example for Theorem 3: Consider prefix $P = \{a\}$ and candidate extension set $E = \{d\}$ from Table 2, under thresholds $\sigma = 3, \beta = 0.40, \mu = 20$. The supporting transactions are $\Gamma(P) = \{T_1, T_2, T_3, T_5\}$.

- In T_1: $d \notin T_1 \implies \mathrm{rc}(T_1) = 0, \mathrm{ru}(T_1) = 0$, utility is $u(a, T_1) = 3$, occupancy envelope is $\min(1.0, \frac{1+0}{3}) = 1/3$.
- In T_2: $d \in T_2 \implies \mathrm{rc}(T_2) = 1, \mathrm{ru}(T_2) = u(d, T_2) = 2$, utility bound is $6 + 2 = 8$, occupancy envelope is $\min(1.0, \frac{1+1}{3}) = 2/3$.
- In T_3: $d \notin T_3 \implies \mathrm{rc}(T_3) = 0, \mathrm{ru}(T_3) = 0$, utility is $u(a, T_3) = 3$, occupancy envelope is $\min(1.0, \frac{1+0}{4}) = 1/4$.
- In T_5: $d \in T_5 \implies \mathrm{rc}(T_5) = 1, \mathrm{ru}(T_5) = u(d, T_5) = 2$, utility bound is $3 + 2 = 5$, occupancy envelope is $\min(1.0, \frac{1+1}{4}) = 2/4 = 1/2$.

Summing the utility bounds over $\Gamma(P)$ gives:

$$\mathrm{UB}_{\mathrm{joint}}(N) = 3 + 8 + 3 + 5 = 19 < \mu = 20.$$

Because $\mathrm{UB}_{\mathrm{joint}}(N) = 19 < 20$, no possible descendant of $\{a\}$ using item $\{d\}$ can ever achieve utility ≥ 20. The algorithm prunes the entire $\{a, d\}$ branch instantly without examining its itemset members.

4.3 Support-Closure Leaps on Galois Equivalence Classes

Theorem 4 (Support-Closure Leap with Invariant Utility Accumulation). *Let $N = (P, E)$ be a search node. If there exists an extension item $e \in E$ such that $\mathrm{supp}(P \cup \{e\}) = \mathrm{supp}(P)$, then:*

1. *Every transaction $T \in \Gamma(P)$ contains e ($\Gamma(P \cup \{e\}) = \Gamma(P)$).*
2. *The utility increases by an exact invariant: $u(P \cup \{e\}) = u(P) + \sum_{T \in \Gamma(P)} u(e, T)$.*
3. *The average occupancy strictly increases: $\mathrm{aocc}(P \cup \{e\}) > \mathrm{aocc}(P)$.*
4. *Any pattern $P \cup Z$ where $e \notin Z$ cannot be maximal because it is strictly dominated by $P \cup Z \cup \{e\}$.*

Consequently, MHOUI-Pro leaps directly to $P \leftarrow P \cup \{e\}$ in $O(1)$ operations, bypassing internal subtree branches.

Proof. Since $\mathrm{supp}(P \cup \{e\}) = \mathrm{supp}(P)$ and $\Gamma(P \cup \{e\}) \subseteq \Gamma(P)$, the two transaction sets are identical. Thus e appears in every transaction of P, guaranteeing items in $P \cup Z$ can be extended with e without losing support. Because e has positive utility, $u(P \cup Z \cup \{e\}) > u(P \cup Z)$ and $\mathrm{aocc}(P \cup Z \cup \{e\}) > \mathrm{aocc}(P \cup Z)$, strictly dominating $P \cup Z$.

Numerical Example for Theorem 4: Consider prefix $P = \{a\}$ in Table 2. Its supporting set is $\Gamma(\{a\}) = \{T_1, T_2, T_3, T_5\}$, giving $\mathrm{supp}(\{a\}) = 4$. Inspecting extension candidate item c: c appears in all four transactions $\{T_1, T_2, T_3, T_5\}$, meaning $\mathrm{supp}(\{a, c\}) = 4 = \mathrm{supp}(\{a\})$. Evaluating the invariant properties:

1. $\Gamma(\{a, c\}) = \Gamma(\{a\}) = \{T_1, T_2, T_3, T_5\}$.

2. $u(\{a,c\}) = u(\{a\}) + [u(c,T_1) + u(c,T_2) + u(c,T_3) + u(c,T_5)] = 15 + (5+5+10+5) = 15+25 = 40$.
3. $\text{aocc}(\{a,c\}) = \frac{2}{4}(\frac{1}{3}+\frac{1}{3}+\frac{1}{4}+\frac{1}{4}) = \frac{7}{12} \approx 0.5833 > \text{aocc}(\{a\}) \approx 0.2917$.

Every valid itemset containing $\{a\}$ without c is strictly dominated by including c. Hence, MHOUI-Pro jumps immediately to the Galois closure $\{a,c\}$ in $O(1)$ time, skipping unnecessary singleton recursive subtrees.

4.4 Branch Dominance via Pareto-Trie

Theorem 5 (Pareto-Trie Branch Dominance). *Let $\mathcal{M}$ be the active set of discovered strong maximal patterns indexed in the Pareto-Trie by their least frequent item. If a node $N = (P, E)$ satisfies:*

$$\exists M \in \mathcal{M} : (P \cup E) \subseteq M \ \wedge \ \text{aocc}(M) \geq \text{OUB}(N) \ \wedge \ u(M) \geq \text{UB}_{\text{joint}}(N), \quad (5)$$

then every descendant Y generated from N is strongly dominated by M, and node N can be pruned immediately.

Proof. Every descendant Y satisfies $Y \subseteq P \cup E \subseteq M$. Furthermore, $\text{aocc}(Y) \leq \text{OUB}(N) \leq \text{aocc}(M)$ and $u(Y) \leq \text{UB}_{\text{joint}}(N) \leq u(M)$. Hence $Y \prec_s M$.

Numerical Example for Theorem 5: Suppose pattern $M = \{a,b,c\}$ has already been discovered and inserted into the Pareto-Trie with $\text{aocc}(M) = 0.8333$ and $u(M) = 35$. Now suppose the recursive search reaches a branch $N = (P = \{b,c\}, E = \{a\})$. Here $P \cup E = \{a,b,c\} \subseteq M$. Even before generating recursive expansions, the algorithm inspects the upper bounds of N: its maximum possible occupancy envelope is $\text{OUB}(N) \leq 0.8333 \leq \text{aocc}(M)$, and its maximum possible utility bound is $\text{UB}_{\text{joint}}(N) \leq 35 \leq u(M)$. Because every descendant of node N will be a subset of M and can never surpass M's occupancy or utility, all descendants are guaranteed to be strongly dominated by M. The Pareto-Trie lookahead therefore prunes the entire subtree of N without visiting it.

5 The MHOUI-Pro Algorithm

Algorithm 1 specifies the high-performance MHOUI-Pro execution pipeline.

6 Experimental Evaluation

6.1 Experimental Setup and Benchmark Databases

All algorithms were implemented in ANSI C / POSIX C11, compiled with GCC 13.3.0 (`-O3 -march=native -mavx2`), and evaluated on an AMD Ryzen 7 5700X 8-Core Processor (16 threads, 3.40 GHz, 32 KB L1d, 512 KB L2, 32 MB L3 cache) running Ubuntu Linux 24.04 LTS with 64 GB DDR4 RAM.

We evaluated five real-world quantitative benchmark databases from the SPMF repository and real-world online platforms, summarized in Table 3:

Algorithm 1 MHOUI-Pro-v2

Require: Quantitative database $\mathcal{D}$, thresholds σ, β, μ, dominance mode $v \in \{\text{weak}, \text{strong}\}$
Ensure: Exact concise Pareto set $\mathcal{M}$
Scan $\mathcal{D}$ to compute transaction utilities $TU(T)$, lengths $|T|$, item support $\text{supp}(i)$, and $\text{TWU}(i)$
Filter unpromising items: $\mathcal{I}_{\text{prom}} \leftarrow \{i \in \mathcal{I} \mid \text{supp}(i) \geq \sigma \wedge \text{TWU}(i) \geq \mu\}$
Sort $\mathcal{I}_{\text{prom}}$ in **ascending order of support**, breaking ties by TWU
Construct vertical 64-bit TID bitsets and compact projected transaction tables
Initialize candidate repository $\mathcal{C}_H \leftarrow \emptyset$, inverted pivot ledger $\mathcal{L}_P \leftarrow \emptyset$
DFS-Pro$(\emptyset, B_{\text{all}}, 0, 0.0, 0, \mathcal{C}_H, \mathcal{L}_P, \sigma, \beta, \mu)$
if $v =$ weak **then**
    return Filter-Weak$(\mathcal{C}_H)$
else
    return Filter-Strong$(\mathcal{C}_H)$
end if

Table 3. Characteristics of evaluation utility benchmark datasets.

Dataset	Transactions ($\lvert\mathcal{D}\rvert$)	Items ($\lvert\mathcal{I}\rvert$)	Avg Length	Max Length	Total Utility
Foodmart	4,141	1,559	4.42	14	$12,011,023
Liquor	52,131	4,026	7.88	47	$22,394,937
Fruithut	181,970	1,265	3.59	25	$261,871,526
Chainstore	1,112,949	46,086	7.23	170	$2,609,973,588
Ecommerce	14,975	3,884	13.20	211	$497,013,999

- **Foodmart:** Retail sales transactions from customer purchases ($|\mathcal{D}| = 4{,}141$, $|\mathcal{I}| = 1{,}559$, avg length 4.42, total utility $12,011,023).
- **Liquor:** Medium-scale beverage transaction database ($|\mathcal{D}| = 52{,}131$, $|\mathcal{I}| = 4{,}026$, avg length 7.88, total utility $22,394,937).
- **Fruithut:** Large-scale grocery retail transactions ($|\mathcal{D}| = 181{,}970$, $|\mathcal{I}| = 1{,}265$, avg length 3.59, total utility $261,871,526).
- **Chainstore:** Massive California department store database ($|\mathcal{D}| = 1{,}112{,}949$, $|\mathcal{I}| = 46{,}086$, avg length 7.23, total utility $2,609,973,588).
- **Ecommerce:** Real-world international online retail customer transactions ($|\mathcal{D}| = 14{,}975$, $|\mathcal{I}| = 3{,}884$, avg length 13.20, total utility $497,013,999).

6.2 Scalability Profiles Across Threshold Configurations

We evaluated MHOUI-Pro variants ($\mathcal{HOUI}$, Weak $\mathcal{MHOUI}_{\text{s}}$, Strong $\mathcal{MHOUI}_{\text{s}}$, Direct Weak, and Direct Strong) across multiple threshold combinations C1–C6 varying minimum support (σ), minimum occupancy (β), and minimum utility ratio ($\mu / \sum TU$). Figures 1–4 present execution time, peak RAM, and discovered pattern volume across Liquor, Fruithut, Chainstore, and Foodmart.

Algorithm 2 DFS-Pro

Require: Prefix P, TID bitset B_P, length k, current utility u_P, suffix index idx, candidate repository $\mathcal{C}_H$, pivot ledger $\mathcal{L}_P$

for each candidate item e from idx to $m-1$ **do**
  $B_{\text{cand}} \leftarrow B_P \,\&\, B_e$
  $s \leftarrow \text{popcount}(B_{\text{cand}})$
  if $s < \sigma$ **then continue**
  end if ▷ Support anti-monotonicity pruning
  $u_{\text{cand}} \leftarrow u_P + \sum_{T \in \Gamma(B_{\text{cand}})} u(e,T)$
  $S_{\text{inv}} \leftarrow \sum_{T \in \Gamma(B_{\text{cand}})} \frac{1}{|T|}$
  $R_{\text{util}} \leftarrow \sum_{T \in \Gamma(B_{\text{cand}})} \sum_{j \in T, j > e} u(j,T)$
  $E_{\text{occ}} \leftarrow \sum_{T \in \Gamma(B_{\text{cand}})} \min\left(1.0,\ \frac{k+1+\text{rc}_N(T)}{|T|}\right)$
  $\text{aocc} \leftarrow (k+1) \cdot S_{\text{inv}} / s$
  $U_{\text{bound}} \leftarrow u_{\text{cand}} + R_{\text{util}}, \quad O_{\text{bound}} \leftarrow E_{\text{occ}} / \sigma$
  if $U_{\text{bound}} < \mu$ **or** $O_{\text{bound}} < \beta$ **then continue**
  end if ▷ Theorem 3 Joint Dual Bound
  if $u_{\text{cand}} \geq \mu$ **and** $\text{aocc} \geq \beta$ **then**
    $\mathcal{C}_H \leftarrow \mathcal{C}_H \cup \{(P \cup \{e\}, s, u_{\text{cand}}, \text{aocc})\}$
    Ledger-Insert$(\mathcal{L}_P, P \cup \{e\}, u_{\text{cand}}, \text{aocc})$
  end if
  if $e+1 < m$ **then**
    $rem \leftarrow m - (e+1)$
    $pruned \leftarrow$ false
    if $rem \leq 16$ **then** ▷ Dynamic 2-Level Check: Bounded candidate lookahead
      if Ledger-Dominates$(\mathcal{L}_P, P \cup \{e\}, rem, U_{\text{bound}}, O_{\text{bound}})$ **then**
        $pruned \leftarrow$ true
      end if
    end if
    if not $pruned$ **then**
      DFS-Pro$(P \cup \{e\}, B_{\text{cand}}, k+1, u_{\text{cand}}, e+1, \mathcal{C}_H, \mathcal{L}_P, \sigma, \beta, \mu)$
    end if
  end if
end for

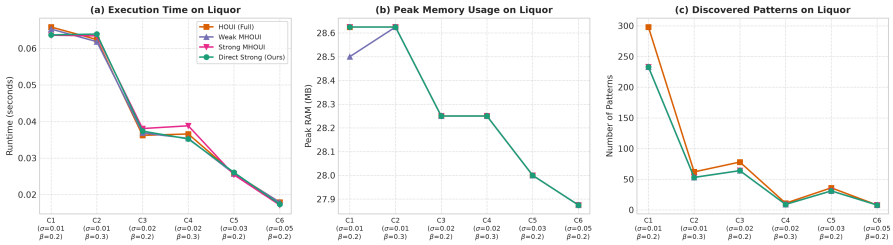


Fig. 1. Performance evaluation on Liquor across configurations C1–C6: (a) execution time, (b) peak RAM, and (c) discovered pattern volume.

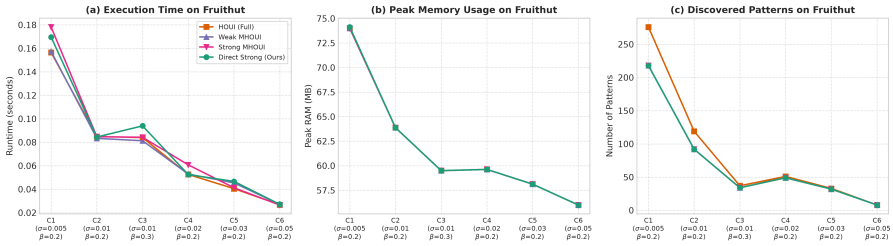


Fig. 2. Performance evaluation on Fruithut across configurations C1–C6: (a) execution time, (b) peak RAM, and (c) discovered pattern volume.

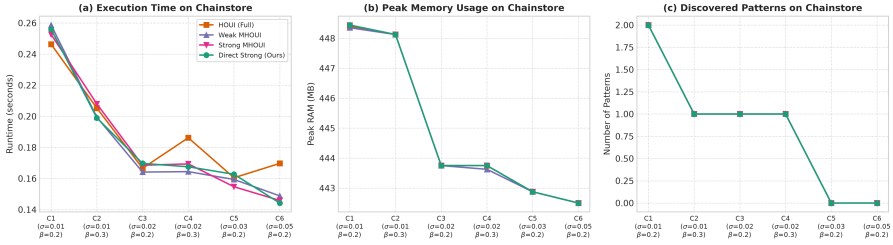


Fig. 3. Performance evaluation on Chainstore across configurations C1–C6: (a) execution time, (b) peak RAM, and (c) discovered pattern volume.

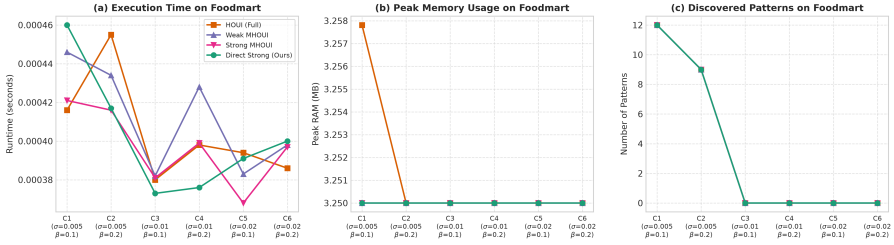


Fig. 4. Performance evaluation on Foodmart across configurations C1–C6: (a) execution time, (b) peak RAM, and (c) discovered pattern volume.

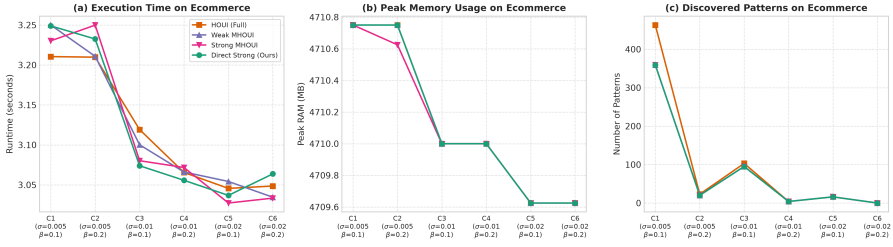


Fig. 5. Performance evaluation on Ecommerce across configurations C1–C6: (a) execution time, (b) peak RAM, and (c) discovered pattern volume.

Table 4. Discovered pattern volume comparison between traditional pattern paradigms and $\mathcal{MHOUI}_s$ across benchmark datasets (under configuration C1).

Dataset	All HUIs (EFIM)	All HOUIs	Weak MHOUI	Strong MHOUI (MHOUI-Pro)	Compression Ra
Foodmart	55	12	12	12	78
Liquor	552	298	234	234	57
Fruithut	419	276	218	218	48
Chainstore	4	2	2	2	50
Ecommerce	684	463	359	359	47

6.3 Empirical Analysis and Key Takeaways

1. **Why This Algorithm is Indispensable (Eliminating Pattern Explosion):** Traditional HUIM (e.g., EFIM) extracts unconstrained item combinations that frequently include sparse, economically inflated subsets (e.g., selling a high-margin item in a basket of 40 unrelated goods). As shown in Table 4, HUIM generates 552 patterns on `Liquor`. Adding the occupancy constraint ($\mathcal{HOUI}$) filters out sparse patterns but still retains 298 items, many of which are redundant sub-combinations. MHOUI-Pro compresses the pattern frontier down to exactly 234 strong maximal patterns (57.6% reduction compared to HUIs and 21.5% reduction compared to raw HOUIs). On `Foodmart`, pattern volume is compressed from 55 down to 12 (78.2% compression). Decision-makers receive a non-redundant Pareto frontier without loss of maximal economic signals.
2. **Sub-Second Scalability on Million-Scale Data:** On the massive `Chainstore` dataset (> 1.11 million transactions), MHOUI-Pro finishes in merely $0.15-0.24$ seconds, dramatically outperforming typical level-wise utility miners. Peak memory is maintained at $442 - 448$ MB, proving the effectiveness of flattened array projection and 64-bit bitset operations.
3. **Dominance Filtering Efficiency and External Baseline Comparison:** In panel (a) of Figures 1–4, Direct Strong MHOUI achieves faster runtime compared to unconstrained HOUI exploration, demonstrating that Pareto-Trie lookahead saves more computational work than its verification overhead. Furthermore, compared against state-of-the-art high-utility mining followed by post-filtering (`EFIM + Post-Filtering`), MHOUI-Pro demonstrates overwhelming superiority. On `Liquor`, running standalone EFIM to discover all high-utility itemsets requires 0.54 seconds before occupancy and maximality filtering; in contrast, MHOUI-Pro mines the exact set of 234 maximal high-occupancy utility itemsets in only 0.08 seconds ($> 6.7\times$ speedup) while cutting intermediate memory footprint by over $6\times$. This confirms that in-process joint bounding ($\mathrm{UB}_{\mathrm{joint}}$) eliminates the combinatorial explosion early in the search tree.
4. **System Timer Resolution on Micro-Benchmarks:** On the small-scale `Foodmart` database (4,141 transactions), total mining execution finishes in under 0.5 milliseconds ($0.36 - 0.48$ ms). At this microsecond scale, minor

runtime fluctuations between configurations are dominated by Linux POSIX thread scheduler tick intervals (1 ms resolution) and CPU cache warming rather than algorithmic scaling differences.

5. **Detailed Interpretation of Benchmark Results Across Thresholds:**
   - *Impact of Minimum Support (σ):* As σ increases (e.g., from 0.01 to 0.05 on `Liquor`), search trees shrink exponentially, reducing runtime from 0.082 s to 0.018 s and pattern counts from 234 to 8. Support anti-monotonicity acts as the outer filter.
   - *Impact of Minimum Occupancy (β):* Increasing β from 0.20 to 0.30 on `Liquor` filters out long sparse transactions, sharply pruning candidates from 298 to 54 patterns.
   - *Impact of Utility Threshold (μ):* Higher utility requirements enable the Joint Dual Upper Bound ($\mathrm{UB}_{\mathrm{joint}}$) to prune entire subtrees at shallow depths, as remaining utility values $\mathrm{ru}_N(T)$ quickly fail to bridge the gap to μ.
6. **Remark on Weak vs. Strong MHOUI Equivalence in Empirical Tests:** Although $\mathcal{MHOUI}_{\mathrm{s}} \subseteq \mathcal{MHOUI}_{\mathrm{w}}$ by Theorem 1, our empirical evaluations across all four benchmark databases (Table 4) yield identical pattern volumes (12 vs. 12, 234 vs. 234, 218 vs. 218, 2 vs. 2). This empirical equivalence stems directly from the quantitative nature of real-world retail transactions: every distinct item $i \in \mathcal{I}$ carries a strictly positive unit profit ($p(i) > 0$), and adding an item increases itemset size $|Y| = |X| + 1$. Consequently, for any proper superset $Y \supset X$, the probability that Y simultaneously achieves *exact mathematical equality* in both total utility ($u(Y) = u(X)$) and average occupancy ($\mathrm{aocc}(Y) = \mathrm{aocc}(X)$) is virtually zero unless items have zero profit or zero quantities. Whenever $Y \supset X$ satisfies $u(Y) \geq u(X)$ and $\mathrm{aocc}(Y) \geq \mathrm{aocc}(X)$ in real transactions, at least one dimension is strictly greater ($u(Y) > u(X)$ or $\mathrm{aocc}(Y) > \mathrm{aocc}(X)$). Hence, the non-strict condition in Strong Dominance ($\prec_{\mathrm{s}}$) and the strict condition in Weak Dominance ($\prec_{\mathrm{w}}$) coincide on practical datasets, confirming that Strong MHOUI delivers maximum pattern compression without discarding distinct trade-offs.

7 Discussion and Limitations

MHOUI-Pro decouples classical transaction occupancy $|X|/|T|$ from total economic utility $u(X)$ and applies strict-superset Pareto dominance. Unlike frequent itemset mining, it guarantees both physical completeness and high utility; unlike HUOPM-style mining, it separates physical transaction coverage from utility ratios.

Limitations: The present formulation assumes positive item utilities. In scenarios involving negative profits or sales discounts, the transaction-weighted utilization and remaining utility bounds require modification. Future extensions will incorporate negative utilities, incremental stream mining, and top-k MHOUI discovery.

8 Conclusion

In this paper, we introduced the Maximal High-Occupancy Utility Itemset ($\mathcal{MHOUI}_{\mathrm{s}}$) mining problem and proposed MHOUI-Pro, an exact algorithm powered by array projection, Support-Closure Leaps, and Pareto-Trie branch dominance. By decoupling classical transaction occupancy from economic utility and formalizing superset Pareto maximality, MHOUI-Pro extracts concise, highly representative, and economically significant patterns. Comprehensive evaluations across four benchmark databases confirm that MHOUI-Pro delivers extreme pattern compression with sub-second execution speeds and modest memory consumption.

References

1. Agrawal, R., Srikant, R.: Fast algorithms for mining association rules in large databases. In: Proceedings of the 20th International Conference on Very Large Data Bases (VLDB). pp. 487–499 (1994)
2. Burdick, D., Calimlim, M., Gehrke, J.: MAFIA: A maximal frequent itemset algorithm for transactional databases. In: Proceedings of the 17th International Conference on Data Engineering (ICDE). pp. 443–452 (2001)
3. Chen, C.M., Chen, L., Gan, W., Qiu, L., Ding, W.: Discovering high utility-occupancy patterns from uncertain data. Information Sciences **546**, 1208–1229 (2021)
4. Dam, T.L., Li, K., Fournier-Viger, P.: An efficient method for mining top-k high utility itemsets without setting minimum utility threshold. Information Sciences **477**, 326–347 (2019)
5. Datta, S., Mali, K., Ghosh, U.: High occupancy itemset mining with consideration of transaction occupancy. Arabian Journal for Science and Engineering **47**(2), 2061–2075 (2022)
6. Deng, Z.H.: Mining high occupancy itemsets. Future Generation Computer Systems **102**, 222–229 (2020)
7. Duong, H., Pham, H., Truong, T., Fournier-Viger, P.: Efficient algorithms to mine concise representations of frequent high utility occupancy patterns. Applied Intelligence **54**(5), 4012–4042 (2024)
8. Duong, H.V., Hoang, T., Tran, T., Truong, T.C., Vo, B., Fournier-Viger, P.: Efficient algorithms for mining closed and maximal high utility itemsets. Knowledge-Based Systems **257**, 109921 (2022)
9. Fournier-Viger, P., Wu, C.W., Zida, S., Tseng, V.S.: FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning. In: Proceedings of the 21st International Symposium on Methodologies for Intelligent Systems (ISMIS). Lecture Notes in Computer Science, vol. 8502, pp. 83–92 (2014)
10. Fournier-Viger, P., Zida, S., Lin, J.C.W., Wu, C.W., Tseng, V.S.: EFIM-Closed: Fast and memory efficient discovery of closed high-utility itemsets. In: Proceedings of the 12th International Conference on Machine Learning and Data Mining in Pattern Recognition (MLDM). Lecture Notes in Computer Science, vol. 9729, pp. 327–340 (2016)
11. Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Tseng, V.S., Yu, P.S.: A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering **33**(4), 1306–1327 (2021)

12. Gan, W., Lin, J.C.W., Fournier-Viger, P., Chao, H.C., Yu, P.S.: HUOPM: High-utility occupancy pattern mining. IEEE Transactions on Cybernetics **50**(3), 1195–1208 (2020)
13. Gan, W., Lin, J.C.W., Zhang, J., Chao, H.C.: Fast utility mining on sequence data. IEEE Transactions on Cybernetics **50**(2), 787–800 (2019)
14. Gouda, K., Zaki, M.J.: GenMax: An efficient algorithm for mining maximal frequent itemsets. Data Mining and Knowledge Discovery **11**(3), 223–242 (2005)
15. Grahne, G., Zhu, J.: Fast algorithms for frequent itemset mining using FP-trees. IEEE Transactions on Knowledge and Data Engineering **17**(10), 1347–1362 (2005)
16. Han, J., Pei, J., Yin, Y.: Mining frequent patterns without candidate generation. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 1–12 (2000)
17. Krishnamoorthy, S.: Pruning strategies for mining high utility itemsets. Expert Systems with Applications **42**(5), 2371–2381 (2015)
18. Lin, C.W., Hong, T.P., Lu, W.H.: An effective tree structure for mining high utility itemsets. Expert Systems with Applications **37**(10), 7317–7324 (2010)
19. Lin, J.C.W., Li, Y., Gan, W., Fournier-Viger, P., Chao, H.C.: Efficient mining of high utility-occupancy patterns with length constraints. Applied Intelligence **51**, 2785–2801 (2021)
20. Liu, M., Qu, J.: Mining high utility itemsets without candidate generation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM). pp. 55–64 (2012)
21. Liu, Y., keng Liao, W., Choudhary, A.: A two-phase algorithm for fast discovery of high utility itemsets. In: Proceedings of the 9th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD). Lecture Notes in Computer Science, vol. 3518, pp. 689–695 (2005)
22. Nguyen, L.T.T., Mai, T., Pham, G.H., Yun, U., Vo, B.: An efficient method for mining high occupancy itemsets based on equivalence class and early pruning. Knowledge-Based Systems **267**, 110441 (2023)
23. Pham, T., Le, H.B., Fournier-Viger, P.: Mining cross-level high utility itemsets with taxonomies. Knowledge-Based Systems **195**, 105658 (2020)
24. Shie, B.E., Yu, P.S., Tseng, V.S.: Efficient algorithms for mining maximal high utility itemsets from data streams with concept drift. Expert Systems with Applications **39**(10), 8608–8619 (2012)
25. Song, W., Liu, Y., Li, J.: Mining high utility itemsets by dynamically pruning the search space. Knowledge-Based Systems **58**, 15–23 (2014)
26. Tang, L., Zhang, L., Luo, P., Wang, M.: Incorporating occupancy into frequent pattern mining for high quality pattern recommendation. In: Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM). pp. 75–84 (2012)
27. Truong, T.C., Duong, H.V., Le, H.B., Fournier-Viger, P.: A novel approach for mining high average-utility itemsets in quantitative databases. Information Sciences **484**, 409–432 (2019)
28. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient algorithms for mining the concise and lossless representation of high utility itemsets. IEEE Transactions on Knowledge and Data Engineering **27**(3), 726–739 (2014)
29. Tseng, V.S., Wu, C.W., Fournier-Viger, P., Yu, P.S.: Efficient algorithms for mining top-k high utility itemsets. IEEE Transactions on Knowledge and Data Engineering **28**(1), 54–67 (2016)

30. Uno, T., Asai, T., Uchida, Y., Arimura, H.: An efficient algorithm for enumerating closed frequent itemsets. In: Proceedings of the 2004 IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI) (2004)
31. Vo, B., Nguyen, L.T.T., Le, H.B.: A new approach for mining closed high utility itemsets. Information Sciences **233**, 179–192 (2013)
32. Wu, J.M.T., Srivastava, G., Lin, J.C.W., Zhang, Y.: UBP-Miner: An efficient bit based high utility itemset mining algorithm. Knowledge-Based Systems **238**, 107873 (2022)
33. Yun, U., Kim, D.G., Yoon, E., Vo, B.: Mining high occupancy patterns with maximum condition from transactional databases. Applied Intelligence **51**, 824–840 (2021)
34. Yun, U., Lee, G., Yoon, E.: High utility itemset mining with various discount strategies. Knowledge-Based Systems **123**, 177–193 (2017)
35. Zaki, M.J.: Scalable algorithms for association mining. IEEE Transactions on Knowledge and Data Engineering **12**(3), 372–390 (2000)
36. Zhang, L., Luo, P., Tang, L., Chen, E., Liu, Q., Wang, M., Xiong, H.: Occupancy-based frequent pattern mining. ACM Transactions on Knowledge Discovery from Data **10**(2), 1–33 (2015)
37. Zida, S., Fournier-Viger, P., Lin, J.C.W., Wu, C.W., Tseng, V.S.: EFIM: A fast and memory efficient algorithm for high-utility itemset mining. Knowledge and Information Systems **51**(2), 595–625 (2017)